

În proiect ai fișierul `algoritmi.h`. E material de citit: are înăuntru cei 15 algoritmi de mai jos, scriși, și o funcție `exemple()` care îi arată rulând. Nu îl incluzi în proiectul tău și nu copiezi din el.

Cum lucrezi, pentru fiecare secțiune:

1. citești algoritmul din `algoritmi.h` până înțelegi ce face;
2. **încizi fișierul** și îl scrii tu, în headerul secțiunii — numele e în titlu;
3. abia după ce merge, îl compari cu materialul;
4. scrii cele 5 funcții în același header, folosind algoritmul tău;
5. în `Source.cpp` incluzi headerul la care lucrezi și chemi funcțiile pe rând.

Reguli:

- **Nu ai voie** să copiezi o funcție din `algoritmi.h` — o scrii tu.
- Dacă o funcție a ta merge și îți trebuie o variantă puțin diferită, scrii **o funcție nouă**, cu nume nou. Nu o modifizi pe cea care merge.
- **Nu ai voie:** `#include <algorithm>`, `sort`, `vector` din bibliotecă, `do...while`.

1. Minim și maxim · `minim-maxim.h`

Scrii întâi tu: `pozMinim`, `pozMaxim`

Amplitudinea

1.1

Scrie o funcție care întoarce diferența dintre cea mai mare și cea mai mică valoare din vector.

```
7 2 9 4 5  -> 7
4 4 4      -> 0
```

Cât de departe sunt

1.2

Scrie o funcție care întoarce câte poziții sunt între minim și maxim. Dacă apar de mai multe ori, se ia prima apariție a fiecăruia.

```
7 2 9 4 5  -> 1
1 8 3 0    -> 2
```

Schimbă-i între ei

1.3

Scrie o funcție care interschimbă minimul cu maximum, lăsând restul vectorului neatins.

```
7 2 9 4 5  -> 7 9 2 4 5
```

Al doilea cel mai mare

1.4

Scrie o funcție care întoarce a doua cea mai mare valoare. Valorile care se repetă se numără o singură dată.

```
7 2 9 4 9  -> 7
3 3 8 1    -> 3
```

Maximul din trei în trei

1.5

Scrie o funcție care afișează maximul fiecărui grup de 3 elemente vecine, luate pe rând de la stânga la dreapta.

```
1 5 2 8 3  ->  5 8 8
4 4 4 4    ->  4 4
```

2. Căutare binară · cautare-binara.h

Scrit în: `cautareBinara`

Apare valoarea?

2.1

Scrie o funcție care spune dacă o valoare apare în vector, folosind căutarea binară. Vectorul primit nu este neapărat ordonat.

```
30 12 45 7 19 , caut 19 -> true
30 12 45 7 19 , caut 20 -> false
```

Câte se regăsesc

2.2

Primești doi vectori. Scrie o funcție care întoarce câte valori din al doilea se găsesc și în primul.

```
v: 5 10 14 19
w: 10 7 19    -> 2
```

Unde ar trebui pusă

2.3

Primești un vector ordonat crescător și o valoare. Scrie o funcție care întoarce poziția pe care ar trebui inserată valoarea ca vectorul să rămână ordonat.

```
2 5 9 14 , pun 7  -> 2
2 5 9 14 , pun 1  -> 0
2 5 9 14 , pun 20 -> 4
```

Cel mai mic dintre cele mari

2.4

Primești un vector ordonat crescător și o valoare x. Scrie o funcție care întoarce cea mai mică valoare din vector mai mare sau egală cu x, sau -1 dacă nu există.

```
2 5 9 14 , x = 6  -> 9
2 5 9 14 , x = 15 -> -1
```

Câte sunt în interval

2.5

Primești un vector ordonat crescător și două valori a și b. Scrie o funcție care întoarce câte elemente sunt între a și b, inclusiv. Fără să parcurgi tot vectorul.

```
2 5 9 14 20 , a = 5 , b = 14 -> 3
```

3. Sortare prin metoda bulelor · `sortare-bule.h`

Scrii întâi tu: `sortareCrescator`, `sortareDescrescator`

Mediana

3.1

Scrie o funcție care întoarce valoarea din mijloc după ordonare. Dacă vectorul are un număr par de elemente, întoarce media celor două din mijloc.

```
7 2 9 4 5    -> 5
32 56 19 42  -> 37
```

Al k-lea cel mai mare

3.2

Scrie o funcție care întoarce a k-a cea mai mare valoare din vector.

```
7 2 9 4 5 , k = 1 -> 9
7 2 9 4 5 , k = 2 -> 7
```

Doar pare în ordine

3.3

Scrie o funcție care ordonează crescător doar valorile pare, lăsând valorile impare pe pozițiile lor.

```
5 8 3 2 9 6  -> 5 2 3 6 9 8
```

E deja ordonat?

3.4

Scrie o funcție care spune dacă vectorul este deja ordonat crescător, fără să-l sortezi.

```
1 3 7 -> true
1 7 3 -> false
```

Câte interschimbări

3.5

Scrie o funcție care întoarce câte interschimbări face metoda bulelor până ordonează vectorul.

```
3 1 2 -> 2
1 2 3 -> 0
```

4. Sortare prin selecție · `sortare-selecție.h`

Scrii întâi tu: `sortareSelectie`

Primele k

4.1

Scrie o funcție care afișează cele mai mici k valori din vector, în ordine crescătoare.

```
7 2 9 4 5 , k = 3 -> 2 4 5
```

Ordonare după ultima cifră

4.2

Scrie o funcție care ordonează crescător vectorul după ultima cifră a fiecărui număr.

```
23 41 15 32 -> 41 32 23 15
```

Ordonare după lungime

4.3

Scrie o funcție care ordonează descrescător vectorul după câte cifre are fiecare număr.

```
7 123 45 6789 -> 6789 123 45 7
```

Valorile distincte, în ordine

4.4

Scrie o funcție care afișează valorile distincte din vector, ordonate crescător.

```
5 2 5 7 2 -> 2 5 7
```

Ordonare cu zerouri fixe

4.5

Scrie o funcție care ordonează crescător vectorul, dar zerourile rămân pe pozițiile pe care erau.

```
0 5 3 0 1 -> 0 1 3 0 5
```

5. Sortare pe interval · `sortare-interval.h`

Scrii întâi tu: `sortareInterval`, `sortareIntervalDescrescator`

Prima jumătate

5.1

Scrie o funcție care ordonează crescător doar prima jumătate a vectorului.

```
9 7 5 3 1 2 -> 5 7 9 3 1 2
```

A doua jumătate, invers

5.2

Scrie o funcție care ordonează descrescător doar a doua jumătate a vectorului.

```
9 7 5 3 1 2 -> 9 7 5 3 2 1
```

Jumătate și jumătate

5.3

Scrie o funcție care ordonează prima jumătate crescător și a doua descrescător.

```
9 7 5 3 1 2 -> 5 7 9 3 2 1
```

Între două poziții

5.4

Scrie o funcție care primește două poziții și ordonează crescător doar elementele dintre ele, inclusiv.

```
9 7 5 3 1 2 , de la 1 la 4 -> 9 1 3 5 7 2
```

Fiecare treime

5.5

Scrie o funcție care împarte vectorul în trei părți egale și ordonează crescător fiecare parte separat.

```
9 7 5 3 1 2 -> 7 9 3 5 1 2
```

6. Munte și vale · `munte-vale.h`

Scrii întâi tu: `sortareMunte`, `sortareVale`

E munte?

6.1

Scrie o funcție care spune dacă vectorul are formă de munte: crește strict până la un vârf, apoi scade strict. Vârful nu are voie să fie primul sau ultimul element.

```
1 4 7 5 2 -> true
1 4 7      -> false
3 3 5 2    -> false
```

E vale?

6.2

Scrie o funcție care spune dacă vectorul are formă de vale: scade strict până la un punct, apoi crește strict.

```
7 3 1 5 8 -> true
1 2 3      -> false
```

Unde e vârful

6.3

Scrie o funcție care întoarce poziția vârfului unui vector în formă de munte.

```
1 4 7 5 2 -> 2
2 9 4      -> 1
```

Fă-l munte

6.4

Scrie o funcție care primește o poziție și rearanjează vectorul ca să fie munte cu vârful pe poziția aia.

```
99 31 54 16 17 63 , varf 3 -> 16 31 54 99 63 17
```

Cel mai lung munte

6.5

Scrie o funcție care întoarce lungimea celei mai lungi porțiuni de elemente vecine care formează un munte.

```
2 1 4 7 3 2 5 -> 5
1 2 3          -> 0
```

7. Interclasare · `interclasare.h`

Scrii întâi tu: `interclasare`

Doi vectori ordonați

7.1

Primești doi vectori ordonați crescător. Scrie o funcție care construiește un al treilea vector ordonat, cu toate elementele lor.

```
a: 1 4 7 9
b: 2 3 8
c: 1 2 3 4 7 8 9
```

Doi vectori neordonați

7.2

Aceeași cerință, dar vectorii primiți nu sunt ordonați.

```
a: 7 1 9
b: 4 2
c: 1 2 4 7 9
```

Ce au în comun

7.3

Primești doi vectori ordonați crescător. Scrie o funcție care construiește vectorul valorilor care apar în amândoi.

```
a: 1 4 7 9
b: 2 4 9
c: 4 9
```

Toate valorile, o singură dată

7.4

Primești doi vectori ordonați crescător. Scrie o funcție care construiește vectorul ordonat al tuturor valorilor, fiecare apărând o singură dată.

```
a: 1 4 7 9
b: 2 4 9
c: 1 2 4 7 9
```

Trei deodată

7.5

Scrie o funcție care interclasează trei vectori ordonați într-unul singur, ordonat.

```
a: 1 5
b: 2 6
c: 3 4
d: 1 2 3 4 5 6
```

8. Ștergerea unui element · `stergere.h`

Scrii întâi tu: `stergereElement`

Șterge o valoare

8.1

Scrie o funcție care șterge din vector toate aparițiile unei valori date.

```
5 2 5 7 5 , sterg 5 -> 2 7
```

Șterge parele

8.2

Scrie o funcție care șterge din vector toate valorile pare.

```
5 8 3 2 9 -> 5 3 9
```

Fără duplicate

8.3

Scrie o funcție care șterge valorile care se repetă, păstrând doar prima apariție a fiecăreia.

```
5 2 5 7 2 2 9 -> 5 2 7 9
```

Sub medie

8.4

Scrie o funcție care șterge elementele mai mici decât media aritmetică a vectorului.

```
4 8 1 7 -> 8 7
```

Aparițiile pare

8.5

Scrie o funcție care șterge valorile care apar de un număr par de ori în vector.

```
5 5 3 7 7 1 -> 3 1
```

9. Inserarea unui element · `inserare.h`

Scrii întâi tu: `inserareElement`

Inserare care păstrează ordinea

9.1

Primești un vector ordonat crescător și o valoare. Scrie o funcție care o pune la locul ei, fără să sortezi la final.

```
2 5 9 14 , pun 7 -> 2 5 7 9 14
```

Fiecare de două ori

9.2

Scrie o funcție care pune după fiecare element încă o copie a lui.

```
1 2 3 -> 1 1 2 2 3 3
```

Zero între

9.3

Scrie o funcție care pune un 0 între oricare două elemente vecine.

```
1 2 3 -> 1 0 2 0 3
```

După fiecare par

9.4

Scrie o funcție care primește o valoare și o inserează după fiecare element par din vector.

```
5 8 3 2 , pun 99 -> 5 8 99 3 2 99
```

Unul de la mine, unul de la tine

9.5

Primești doi vectori. Scrie o funcție care construiește un vector cu primul element din a, primul din b, al doilea din a, al doilea din b, și tot așa. Ce rămâne se pune la final.

```
a: 1 2 3  
b: 7 8  
c: 1 7 2 8 3
```

10. Cea mai lungă secvență de valori egale · `secventa-egale.h`

Scrii întâi tu: `ceaMaiLungaSecventaEgale`

Cât de lungă

10.1

Scrie o funcție care întoarce lungimea celei mai lungi secvențe de elemente vecine egale.

```
5 5 1 1 1 1 2 2 -> 4
```

Care valoare

10.2

Scrie o funcție care întoarce valoarea care formează cea mai lungă secvență de elemente vecine egale.

```
5 5 1 1 1 1 2 2 -> 1
```

Câte secvențe

10.3

Scrie o funcție care întoarce în câte secvențe de valori egale se împarte vectorul.

```
5 5 1 1 1 2 -> 3  
1 2 3 -> 3
```

Cea mai lungă urcare

10.4

Scrie o funcție care găsește cea mai lungă secvență de elemente vecine strict crescătoare și întoarce poziția de început și lungimea.

```
2 7 1 3 5 9 4 -> pozitia 2 , lungimea 4
5 4 3          -> pozitia 0 , lungimea 1
```

Aceeași paritate

10.5

Scrie o funcție care întoarce lungimea celei mai lungi secvențe de elemente vecine care au toate aceeași paritate.

```
3 7 2 4 6 5 -> 3
```

11. Poziția unei secvențe · `pozitie-secventa.h`

Scrii întâi tu: `pozitieSecventa`

Unde începe

11.1

Primești doi vectori. Scrie o funcție care întoarce poziția din primul de unde începe al doilea, sau -1 dacă nu apare.

```
v: 8 5 8 5 2 3 10 7
y: 5 2 3          -> 3
```

De câte ori

11.2

Scrie o funcție care întoarce de câte ori apare al doilea vector în primul.

```
v: 1 2 1 2 1 2
y: 1 2          -> 3
```

Ultima apariție

11.3

Scrie o funcție care întoarce poziția ULTIMEI apariții a celui de-al doilea vector în primul.

```
v: 1 2 1 2 1 2
y: 1 2          -> 4
```

Nu neapărat vecine

11.4

Scrie o funcție care spune dacă toate elementele celui de-al doilea vector apar în primul, în aceeași ordine, dar nu neapărat unul lângă altul.

```
v: 8 5 8 5 2 3
y: 8 2 3      -> true
y: 3 5        -> false
```

Cea mai lungă bucată comună

11.5

Primești doi vectori. Scrie o funcție care întoarce lungimea celei mai lungi secvențe de elemente vecine care apare în amândoi.

```
a: 1 4 7 9 2
b: 8 4 7 9    -> 3
```

12. Secvență cu sumă dată · `secventa-suma.h`

Scrii întâi tu: `existaSecventaCuSuma`

Există?

12.1

Scrie o funcție care spune dacă există o secvență de elemente vecine cu o sumă dată și, dacă da, de unde începe și cât e de lungă. Elementele sunt pozitive.

12 10 15 7 17 , suma 32 -> pozitia 1 , lungimea 3

Și cu numere negative

12.2

Aceeași cerință, dar vectorul poate conține și numere negative.

4 -2 3 1 -5 6 , suma 6 -> pozitia 0 , lungimea 4
-3 2 -1 , suma 0 -> nu exista

Cea mai scurtă

12.3

Scrie o funcție care găsește cea mai scurtă secvență de elemente vecine cu o sumă dată.

1 2 3 4 2 4 , suma 6 -> pozitia 3 , lungimea 2

Câte sunt

12.4

Scrie o funcție care întoarce câte secvențe de elemente vecine au o sumă dată.

1 2 3 4 2 4 , suma 6 -> 3

Cel mult k elemente

12.5

Scrie o funcție care întoarce cea mai mare sumă a unei secvențe de elemente vecine care are cel mult k elemente.

2 9 1 8 3 , k = 2 -> 11

13. Suma maximă a unei secvențe · `suma-maxima.h`

Scrii întâi tu: `sumaMaximaSecventa`

Suma maximă

13.1

Scrie o funcție care întoarce cea mai mare sumă a unei secvențe de elemente vecine, plus poziția de început și lungimea ei.

-4 1 -5 1 4 -2 2 3 -4 -> suma 8 , pozitia 3 , lungimea 5

Suma minimă

13.2

Scrie o funcție care întoarce cea mai mică sumă a unei secvențe de elemente vecine.

-4 1 -5 1 4 -2 2 3 -4 -> -8

Exact k elemente

13.3

Scrie o funcție care întoarce cea mai mare sumă a unei secvențe de exact k elemente vecine.

2 9 1 8 3 , k = 3 -> 18

Cel puțin două

13.4

Scrie o funcție care întoarce cea mai mare sumă a unei secvențe de elemente vecine care are cel puțin 2 elemente.

```
5 -9 6 -1 7 -> 12
```

Două secvențe

13.5

Scrie o funcție care întoarce cea mai mare sumă care se poate obține din două secvențe de elemente vecine care nu se suprapun.

```
3 -2 5 -8 4 6 -> 16
```

14. Frecvența valorilor · `frecventa-valori.h`

Scrii întâi tu: `frecventaValori`

Cel mai frecvent

14.1

Scrie o funcție care întoarce valoarea care apare de cele mai multe ori. La egalitate, o întoarce pe cea mai mică. Elementele sunt între 0 și 100.

```
5 5 5 2 2 7 -> 5
4 4 9 9 1 -> 4
```

Unicatele

14.2

Scrie o funcție care afișează, în ordine crescătoare, valorile care apar o singură dată în vector.

```
5 5 3 7 7 1 -> 1 3
```

Câte valori diferite

14.3

Scrie o funcție care întoarce câte valori distincte are vectorul.

```
5 2 5 7 2 -> 3
```

Prima care se repetă

14.4

Scrie o funcție care întoarce prima valoare care apare a doua oară, parcurgând vectorul de la stânga la dreapta. Dacă nu există, întoarce -1.

```
5 2 7 2 5 -> 2
1 2 3 -> -1
```

Majoritarul

14.5

Scrie o funcție care întoarce valoarea care apare de mai mult de jumătate din numărul de elemente, sau -1 dacă nu există.

```
3 3 4 3 3 -> 3
1 1 2 2 -> -1
```

15. Frecvența cifrelor · frecventa-cifre.h

Scrii întâi tu: `frecventaCifre`

Cifra cea mai des întâlnită

15.1

Scrie o funcție care întoarce cifra care apare de cele mai multe ori în toate numerele din vector, luate împreună.

```
21 42 21 -> 2
```

Cifrele care lipsesc

15.2

Scrie o funcție care afișează cifrele care nu apar în niciun număr din vector.

```
21 42 21 -> 0 3 5 6 7 8 9
```

Câte conțin cifra

15.3

Scrie o funcție care primește o cifră și întoarce câte numere din vector o conțin.

```
70 17 23 7 , cifra 7 -> 3
```

Cele mai variate cifre

15.4

Scrie o funcție care întoarce numărul din vector care are cele mai multe cifre diferite.

```
111 1234 55 907 -> 1234
```

Numere prietene

15.5

Două numere sunt prietene dacă sunt formate exact din aceleași cifre, în orice ordine și cu aceleași repetări. Scrie o funcție care întoarce câte perechi de numere prietene are vectorul.

```
21 12 34 43 21 -> 4
```

Pentru fiecare funcție pe care o scrii, verific-o pe:

1. un vector cu **un singur** element;
2. un vector în care **toate** valorile sunt egale;
3. cazul în care răspunsul e **primul** element, și cel în care e **ultimul**;
4. o valoare care **nu există** în vector.